

BudgetBeacon — Project Documentation

AI-powered business cost estimation with confidence scoring, multi-currency, PDF export, and smart insights.

Table of Contents

- 1. [Project Overview](#)
- 2. [Architecture](#)
- 3. [Features](#)
- 4. [API Reference](#)
- 5. [Machine Learning Model](#)
- 6. [Database Schema](#)
- 7. [Frontend Pages & Components](#)
- 8. [Authentication & Security](#)
- 9. [Setup & Installation](#)
- 10. [Troubleshooting](#)

1. Project Overview

BudgetBeacon is a full-stack AI platform targeting small and medium enterprises (SMEs) in Tanzania and East Africa. It predicts total business costs from user-provided inputs and returns a **confidence score** indicating estimate reliability, plus actionable **smart insights** to help users optimize their spending.

Target Audience

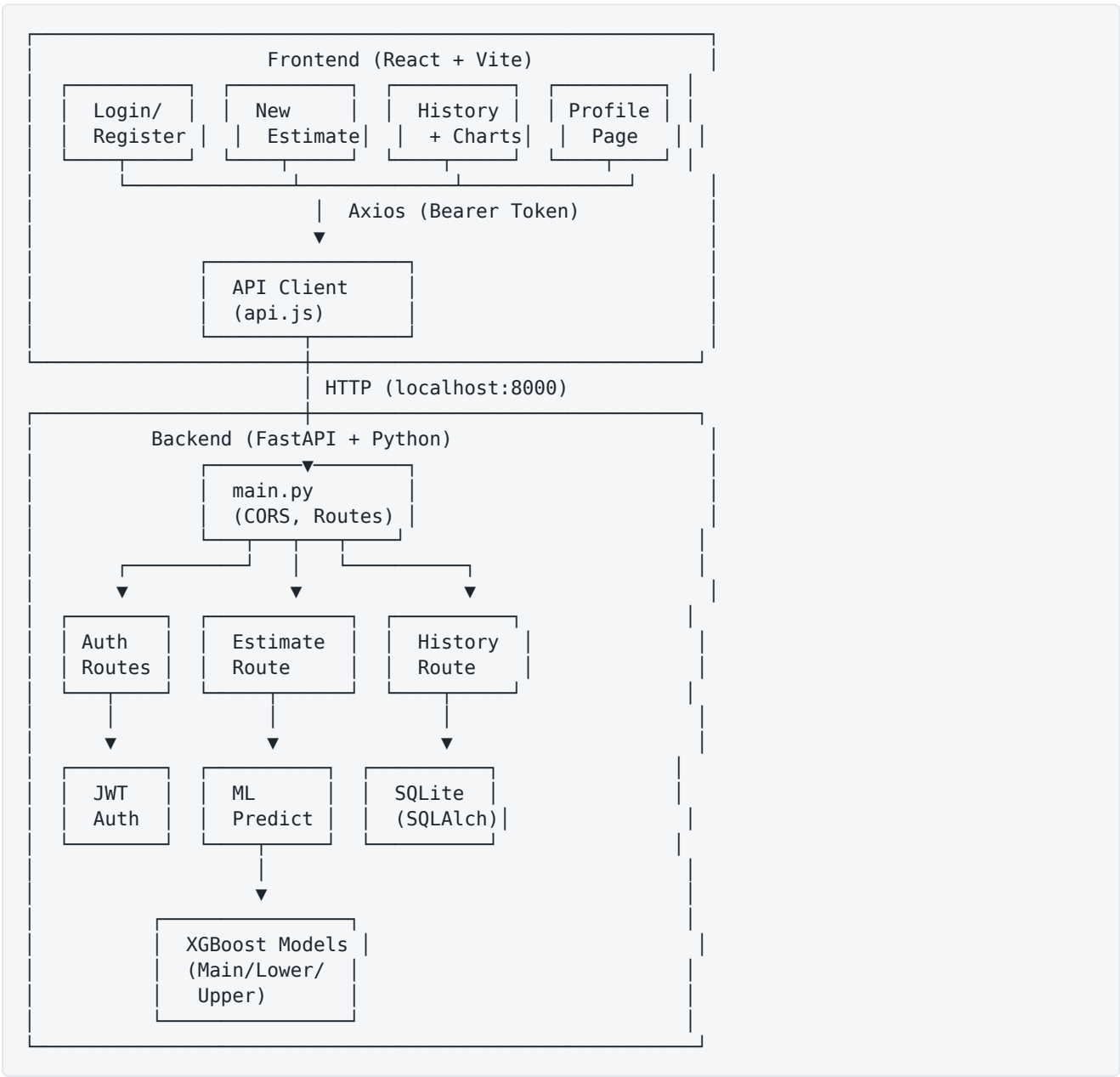
- SME owners, entrepreneurs, and startup founders in East Africa
- Users without deep financial expertise who need quick, reliable cost estimates
- Mobile-first users who can access via USSD on basic phones

Core Value Proposition

Problem	Solution
Business owners struggle to estimate costs accurately	AI model trained on cost patterns predicts total costs
Unknown reliability of estimates leads to poor planning	Confidence score (0-100%) + risk level tells users how much to trust the result

Problem	Solution
Hard to know which costs are out of line	Smart Insights compare inputs against industry/location averages
No easy way to share estimates with customers	PDF quote export + WhatsApp-ready data
Multi-currency confusion for cross-border trade	TZS / USD / UGX support with automatic conversion
No history tracking	Dashboard with charts, stats, and downloadable history

2. Architecture



Tech Stack

Layer	Technology	Role
Frontend	React 19 + Vite 8	SPA with client-side routing
Styling	Tailwind CSS 4 + Dasher design system	Responsive UI with dark/light theme
Charts	Recharts 3	Cost trends, business mix, confidence distribution
HTTP Client	Axios	API calls with auto Bearer token injection
PDF	html2canvas + jsPDF	Client-side PDF quote generation
Backend	FastAPI 0.115 (Python 3.12+)	REST API with auto-docs (Swagger)
ORM	SQLAlchemy 2.0	Database abstraction
Database	SQLite (dev) / PostgreSQL (prod)	Data persistence
Auth	bcrypt + python-jose (JWT)	Password hashing + stateless tokens
ML Engine	XGBoost 2.1 (XGBRegressor)	Three-model quantile regression
Data	numpy + pandas + joblib	Data generation, model serialization

3. Features

3.1 User Authentication

- **Register:** Email, password, optional business name and phone
- **Login:** Email + password, returns JWT token (24h expiry)
- **Profile:** View and edit business name, phone number
- **Protected Routes:** All `/dashboard/*` pages require authentication; 401 redirects to `/login`
- **Logout:** Frontend-only token removal (stateless JWT)

3.2 Cost Estimation Wizard

A 4-step onboarding form:

Step	Name	Fields
1	Business	Business type (5 options with icons), Location (7 options as pills)
2	Costs	Material Cost, Transport Cost, Labor Cost (with currency selector)
3	Production	Production Days, Quantity
4	Review	Full summary with rough total, Submit button

Each step has a visual indicator, progress tracking, and fade-in animations. The form validates all inputs before submission.

3.3 Multi-Currency Support

Users can choose among three currencies:

Currency	Code	Symbol	Rate (vs TZS)
Tanzanian Shilling	TZS	TSh	1 (base)
US Dollar	USD	\$	1 USD = 2,500 TZS
Ugandan Shilling	UGX	USh	1 UGX ≈ 0.67 TZS

Flow:

- 1. User selects a currency before entering costs
- 2. Inputs are displayed and entered in the chosen currency
- 3. Before the API call, the frontend converts all cost values to TZS (the model is trained on TZS data)
- 4. The backend stores both the TZS costs and the original currency code
- 5. The response includes the predicted cost in TZS, and the frontend converts it back to the user's currency
- 6. History and all displays remain currency-aware

Location of conversion logic: `frontend/src/utils/currency.js`

3.4 Savings Insights & Tips

After each estimate, the backend generates contextual tips by comparing user inputs against pre-computed averages for the selected business type and location.

Tip Categories:

Condition	Example Tip
Material cost >30% above average	"Material costs are X% above average. Consider bulk purchasing."
Transport cost >30% above average	"Transport costs are X% above average. Consolidating shipments could help."
Labor cost >30% above average	"Labor costs are X% above average. Review workforce allocation."
Any cost <30% below average	"X costs are below average — your approach looks efficient."
Production days >30% above average	"Production timeline is X% longer than typical. Process optimization could help."
Confidence <50%	"Try using cost values closer to typical ranges for your business."
Confidence ≥90%	"Confidence is very high. This estimate is reliable for planning."

Implementation: `backend/app/ml/tips.py` — hardcoded average values derived from the synthetic training data generation formula.

3.5 PDF Quotation Export

Users can download a professional PDF quote for any estimate:

- **ResultCard:** "PDF" button appears after estimate submission
- **History:** Each row has a PDF download icon
- **Content:** Branded template with logo, business info, breakdown table, total, confidence, risk, date, and footer
- **Generation:** Client-side using `html2canvas` + `jsPDF` (no server load)
- **Template:** Hidden styled div captured to canvas, rendered to A4 PDF

3.6 Dashboard (Home)

Component	Description
3 Stat Cards	Total Estimates, Avg Confidence %, Avg Estimated Cost
Latest Estimate Card	Quick summary of most recent estimate
Cost Trends Chart	Area chart of last 10 estimates
Business Mix Chart	Donut/ pie chart by business type
Confidence Distribution	Horizontal bar chart (High/Medium/Low buckets)
Feature Cards	"Accurate Predictions", "Confidence Scores", "Fast & Accessible"
How It Works	4-step visual guide to using the platform

All charts are built with **Recharts** and respond to dark/light theme via CSS variables.

3.7 Estimate History

- Sortable table with columns: ID, Business Type, Predicted Cost, Confidence, Risk, Date
- Stats cards showing totals, averages
- Bar chart of recent estimates
- PDF download per row
- Empty state with link to create first estimate

3.8 Dark / Light Theme

- Toggle via moon/sun icon in navbar
- Persisted in `localStorage`
- Uses `data-bs-theme` attribute on `<html>` matching Bootstrap 5.3 / Dasher convention
- All colors defined as CSS variables in `:root` and `[data-bs-theme="dark"]`
- Charts and components automatically respond to theme changes

3.9 USSD Webhook (Mobile)

Endpoint: `POST /api/ussd/webhook`

A state-machine-based USSD handler for integration with Africa's Talking:

Step	User Input	Response
0	(none)	"Select business type: 1. Agriculture 2. Retail..."
1	1-5	"Enter material cost (Tsh):"
2	number	"Enter transport cost (Tsh):"
3	number	"Enter labor cost (Tsh):"
4	number	"Enter production days:"
5	All inputs	Final result: cost, confidence, risk

Limitations (dev): Sessions stored in-memory, location defaults to Dar es Salaam, quantity defaults to 1.

3.10 Keyboard Search (Ctrl+K)

- Press `Ctrl+K` or `Cmd+K` to open a search modal
- Filters navigation items by label + keywords
- Arrow keys navigate results, Enter selects
- Click backdrop or Escape to close

3.11 Notifications Panel

- Bell icon in navbar with unread badge count
- Two default welcome/update notifications
- "Mark all read" clears the list
- Clicking individual notification marks it as read and removes it

4. API Reference

4.1 Authentication

Method	Endpoint	Auth	Description
POST	<code>/api/auth/register</code>	No	Create account (email, password, optional business_name, phone)
POST	<code>/api/auth/login</code>	No	Login, returns JWT token + user object
GET	<code>/api/auth/me</code>	Yes	Get current user profile
PUT	<code>/api/auth/me</code>	Yes	Update business_name and/or phone

Register Request:

```
{
  "email": "user@example.com",
  "password": "securepass123",
  "business_name": "My Business",
  "phone": "+255712345678"
}
```

Login Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": {
    "id": 1,
    "email": "user@example.com",
    "business_name": "My Business",
    "phone": "+255712345678"
  }
}
```

4.2 Estimates

Method	Endpoint	Auth	Description
POST	/api/estimate	Yes	Submit inputs, get predicted cost + confidence + tips

Request:

```
{
  "business_type": "agriculture",
  "material_cost": 500000,
  "transport_cost": 150000,
  "labor_cost": 200000,
  "production_days": 14,
  "quantity": 100,
  "location": "Dar es Salaam",
  "currency": "TZS"
}
```

business_type options: agriculture, retail, manufacturing, transport, services

location options: Dar es Salaam, Arusha, Mwanza, Dodoma, Zanzibar, Mbeya, Other

currency options: TZS, USD, UGX

Response:

```
{
  "predicted_cost": 1214468.39,
  "confidence_pct": 87.9,
  "risk_level": "Low",
  "breakdown": {
    "material": 500000.0,
    "transport": 150000.0,
    "labor": 200000.0,
    "overhead_estimated": 364468.39
  },
  "timestamp": "2026-06-30T10:30:00",
  "currency": "TZS",
}
```

```

"tips": [
  "Transport costs are 25% above average for Agriculture in Dar es Salaam. Consolidating ship
  "Confidence is very high. This estimate is reliable for budgeting and planning."
]
}

```

4.3 History

Method	Endpoint	Auth	Description
GET	/api/history	Yes	List user's past estimates (supports ?limit=N)

Response: Array of:

```

{
  "id": 1,
  "business_type": "agriculture",
  "predicted_cost": 1214468.39,
  "confidence_pct": 87.9,
  "risk_level": "Low",
  "currency": "TZS",
  "created_at": "2026-06-30T10:30:00"
}

```

4.4 USSD Webhook

Method	Endpoint	Auth	Description
POST	/api/ussd/webhook	No	Africa's Talking USSD callback

Request: Form-encoded (sessionId , serviceCode , phoneNumber , text)

Response: CON ... (continue) or END ... (terminate) formatted for USSD

4.5 Health

Method	Endpoint	Description
GET	/	Root status
GET	/health	Health check

5. Machine Learning Model

5.1 Algorithm: XGBoost Quantile Regression

Three separate XGBRegressor models trained on 5,000 synthetic records:

Model	Objective	alpha	Purpose
Main	reg:squarederror	—	Predict expected (mean) total cost
Lower	reg:quantileerror	0.1	10th percentile (best-case lower bound)
Upper	reg:quantileerror	0.9	90th percentile (worst-case upper bound)

Hyperparameters:

- n_estimators : 200 (trees)
- max_depth : 5
- learning_rate : 0.1
- random_state : 42

5.2 Input Features (7)

Feature	Type	Range
business_type_idx	Categorical (0-4)	agriculture=0, retail=1, manufacturing=2, transport=3, services=4
location_idx	Categorical (0-6)	Dar es Salaam=0, Arusha=1, Mwanza=2, Dodoma=3, Zanzibar=4, Mbeya=5, Other=6
material_cost	Float (TZS)	50,000 - 2,000,000
transport_cost	Float (TZS)	10,000 - 500,000
labor_cost	Float (TZS)	50,000 - 1,000,000
production_days	Integer	1 - 90
quantity	Integer	1 - 999

5.3 Prediction Logic

```

<div style="page-break-before: always"></div>
# 1. Encode categorical inputs to integer indices
business_type_idx = BUSINESS_TYPES.index(data.business_type)
location_idx = LOCATIONS.index(data.location)

<div style="page-break-before: always"></div>
# 2. Predict with all 3 models
X = [[business_type_idx, location_idx, material_cost, transport_cost,
      labor_cost, production_days, quantity]]
predicted = main_model.predict(X)
lower = lower_model.predict(X)
upper = upper_model.predict(X)

<div style="page-break-before: always"></div>
# 3. Calculate confidence
interval_width = max(0, upper - lower)
confidence = max(0, min(100, (1 - interval_width / predicted) * 100))

<div style="page-break-before: always"></div>
# 4. Determine risk level
if confidence >= 75: risk = "Low"

```

```
elif confidence >= 50: risk = "Medium"
else: risk = "High"

<div style="page-break-before: always"></div>
# 5. Calculate overhead breakdown
total_inputs = material + transport + labor
overhead = max(0, predicted - total_inputs)
```

5.4 Synthetic Data Generation

Target variable formula:

```
overhead = (material + transport + labor) × random(0.05, 0.25)
base_cost = material + transport + labor + overhead
actual_cost = (base_cost × business_multiplier × location_multiplier) + noise(5%)
```

Multipliers:

Business Type	Multiplier
Agriculture	1.0
Retail/Trade	0.8
Manufacturing	1.2
Transport	0.9
Services	0.7

Location	Multiplier
Dar es Salaam	1.1
Arusha	1.0
Mwanza	0.95
Dodoma	0.85
Zanzibar	1.05
Mbeya	0.9
Other	0.8

R² Score: ~0.98 on synthetic data.

5.5 Retraining

To retrain with real data, replace generate_synthetic_data() in backend/app/ml/train.py with a CSV loader:

```
python -m app.ml.train
```

Models are saved to backend/models/ as .pkl files (loaded via joblib).

6. Database Schema

Table: users

Column	Type	Constraints
id	Integer	PK, auto-increment, indexed
email	String	UNIQUE, NOT NULL, indexed
password_hash	String	NOT NULL (bcrypt hash)
business_name	String	Nullable
phone	String	Nullable
created_at	DateTime	

Relationships: User has many Estimates

Table: estimates

Column	Type	Constraints
id	Integer	PK, auto-increment, indexed
user_id	Integer	FK → users.id , nullable
business_type	String	NOT NULL
material_cost	Float	
transport_cost	Float	
labor_cost	Float	
production_days	Integer	
quantity	Integer	
location	String	
currency	String	Default "TZS"
predicted_cost	Float	
confidence_pct	Float	
risk_level	String	
created_at	DateTime	

Default DB: SQLite (`sqlite:///./oraph.db`), configurable via `DATABASE_URL` env var.

7. Frontend Pages & Components

Page Map

```

/login          → Login page (public)
/register       → Register page (public)
/              → Redirects to /dashboard
/dashboard      → Home / Dashboard (protected)
/dashboard/estimate → New Estimate wizard (protected)
/dashboard/history → History table + charts (protected)
/dashboard/profile → Profile editor (protected)

```

Component Tree

```

App.jsx
├── AuthProvider (Context)
├── Router
│   ├── /login → Login.jsx
│   ├── /register → Register.jsx
│   └── /dashboard/* → Layout (ProtectedRoute)
│       ├── Sidebar (collapsible 250px/60px)
│       │   ├── Brand logo
│       │   ├── Nav items (Dashboard, New Estimate, History, Profile)
│       │   └── Profile card → /dashboard/profile
│       ├── Navbar
│       │   ├── Sidebar toggle
│       │   ├── Theme toggle (dark/light)
│       │   ├── Search button → Ctrl+K modal
│       │   ├── Notifications bell → dropdown
│       │   ├── User avatar → /dashboard/profile
│       │   └── Logout button
│       └── <Outlet>
│           ├── Home.jsx (index route)
│           │   ├── 3 Stat cards
│           │   ├── Latest estimate card
│           │   ├── Cost Trends (AreaChart)
│           │   ├── Business Mix (PieChart)
│           │   ├── Confidence Distribution (BarChart)
│           │   └── Feature cards + How It Works
│           ├── NewEstimate.jsx
│           │   ├── Step indicator (4 steps)
│           │   ├── Step 1: Business type + Location selector
│           │   ├── Step 2: Cost inputs + Currency selector
│           │   ├── Step 3: Production days + Quantity
│           │   ├── Step 4: Review + Submit
│           │   └── On submit → ResultCard.jsx
│           │       ├── Predicted cost display
│           │       ├── Confidence bar + Risk badge
│           │       ├── Cost breakdown table
│           │       ├── Smart Insights section
│           │       ├── Insight message
│           │       └── New Estimate + Download PDF buttons
│           ├── History.jsx
│           │   ├── 3 Stat cards
│           │   ├── Bar chart of recent estimates
│           │   └── Table with PDF download per row
│           └── Profile.jsx
│               └── Avatar + email (read-only)

```

└─ Business name (editable)
└─ Phone (editable)

Utility Modules

File	Purpose
src/services/api.js	Axios instance with auth interceptor
src/context/AuthContext.jsx	User state + token management
src/utlis/currency.js	Conversion rates + formatting

8. Authentication & Security

Password Hashing

Direct bcrypt (bypassing deprecated `passlib`):

```
import bcrypt
hash = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
match = bcrypt.checkpw(plain.encode(), hashed.encode())
```

JWT Tokens

- Algorithm: HS256
- Expiry: 24 hours
- Secret: Configurable via `JWT_SECRET` env var (default: development-only key)
- Payload: `{"sub": user_id, "exp": expiry_timestamp}`

Frontend Security

- Token stored in `localStorage`
- Axios request interceptor adds `Authorization: Bearer <token>` to every request
- Axios response interceptor on 401: clears `localStorage`, redirects to `/login`
- `ProtectedRoute` component wraps all dashboard routes

9. Setup & Installation

Prerequisites

- Python 3.12+ (3.14 not supported by `pydantic-core` /PyO3)
- Node.js 20+
- npm 9+

Quick Start (Linux / macOS)

```
<div style="page-break-before: always"></div>
# 1. Backend setup
cd BudgetBeacon/backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

<div style="page-break-before: always"></div>
# 2. Train ML model (first time only)
python -m app.ml.train

<div style="page-break-before: always"></div>
# 3. Start backend
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload &
<div style="page-break-before: always"></div>
# Runs at http://localhost:8000 | Swagger: http://localhost:8000/docs

<div style="page-break-before: always"></div>
# 4. Frontend setup
cd ../frontend
npm install
npm run dev
<div style="page-break-before: always"></div>
# Runs at http://localhost:5173
```

Quick Start (Windows PowerShell)

```
<div style="page-break-before: always"></div>
# 1. Backend setup
cd BudgetBeacon\backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

<div style="page-break-before: always"></div>
# 2. Train ML model (first time only)
python -m app.ml.train

<div style="page-break-before: always"></div>
# 3. Start backend
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

<div style="page-break-before: always"></div>
# 4. Frontend setup (new terminal)
cd ..\frontend
npm install
npm run dev
```

Environment Variables

Variable	Default	Description
DATABASE_URL	sqlite:///./oraph.db	Database connection string
JWT_SECRET	budgetbeacon-secret-key-...	JWT signing secret (change in production)

10. Troubleshooting

Problem	Solution
<code>pip install</code> fails on Debian/Ubuntu	Add <code>--break-system-packages</code> flag
Python 3.14 <code>pydantic-core</code> error	Use Python 3.12 or 3.13
<code>passlib + bcrypt >=4.1</code> incompatibility	Already resolved — using direct <code>bcrypt</code> API
"sqlite3.OperationalError: attempt to write a readonly database"	Kill old server process (<code>fuser -k 8000/tcp</code>) and delete <code>oraph.db</code>
Database schema missing <code>currency</code> column	Delete <code>oraph.db</code> and restart (SQLite auto-creates tables)
"Address already in use" on port 8000/5173	Kill existing process or use a different port
Frontend shows blank page after login	Check browser console for CORS errors; ensure backend is running on port 8000
ML models fail to load	Run <code>python -m app.ml.train</code> to regenerate <code>.pkl</code> files
Estimates return very low confidence	Ensure cost values are in TZS (not USD without conversion) or use the frontend currency selector
PDF download doesn't work	Ensure <code>html2canvas</code> and <code>jspdf</code> are installed (<code>npm install</code>)

Generated: 2026-06-30 — For the latest version, see the project repository.